



MACHINE LEARNING CON PYTHON

TEMA 7. ENSAMBLAJE DE
MODELOS (ENSEMBLES) Y
APRENDIZAJE NO SUPERVISADO



Contenidos Tema 7

- / 1. Ensamblaje de Modelos
- / 2. Optimización de Modelos: ajuste de hiperparámetros
- / 3. ML No supervisado: clustering y reducción de dimensionalidad

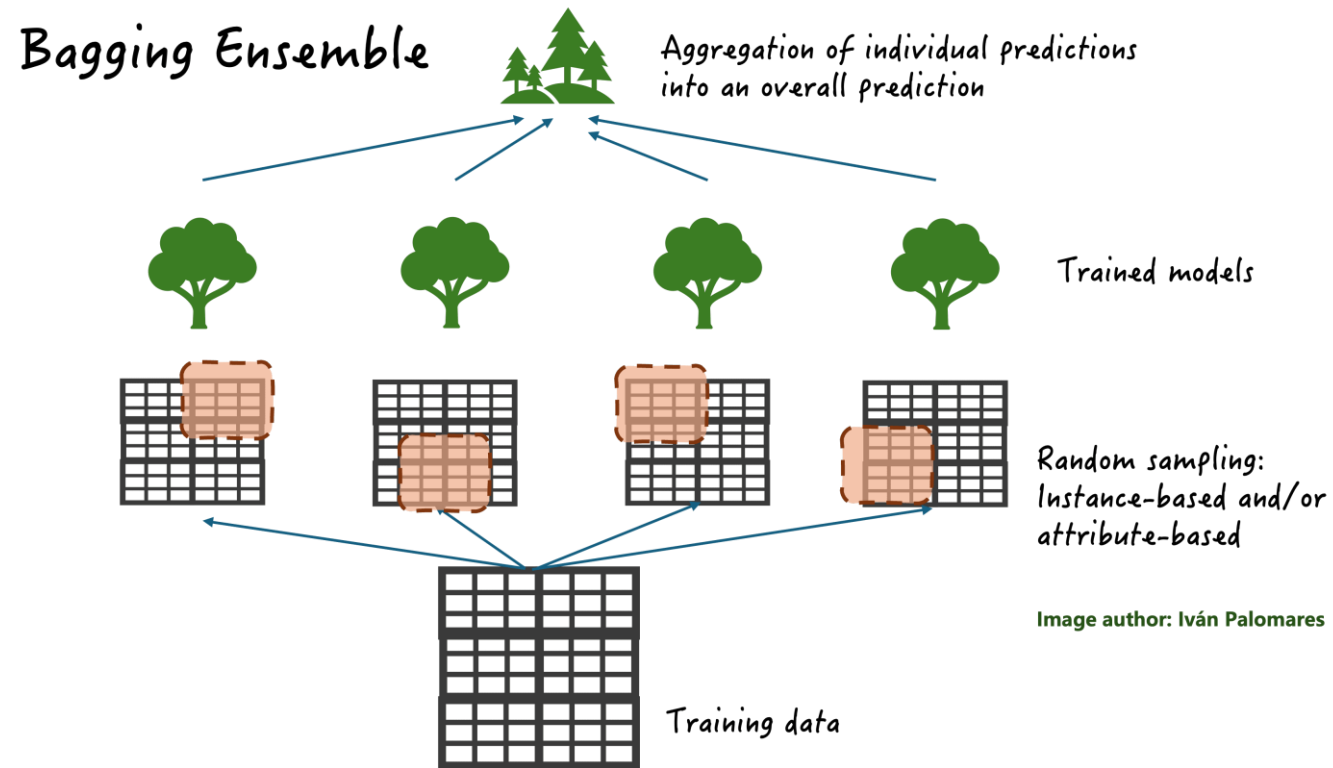
ENLACE A GITHUB CON LOS MATERIALES:

<https://github.com/IconoTC/Machine-Learning-con-PYTHON>

REPOSITORIO DE DATASETS PARA EL CURSO: <https://github.com/gakudo-ai/open-datasets>

Ensembles

Combinar múltiples modelos en una sola tarea predictiva



BAGGING

Se entrenan varios modelos en paralelo, cada uno con una muestra aleatoria del conjunto de entrenamiento.

Se agregan las predicciones individuales:

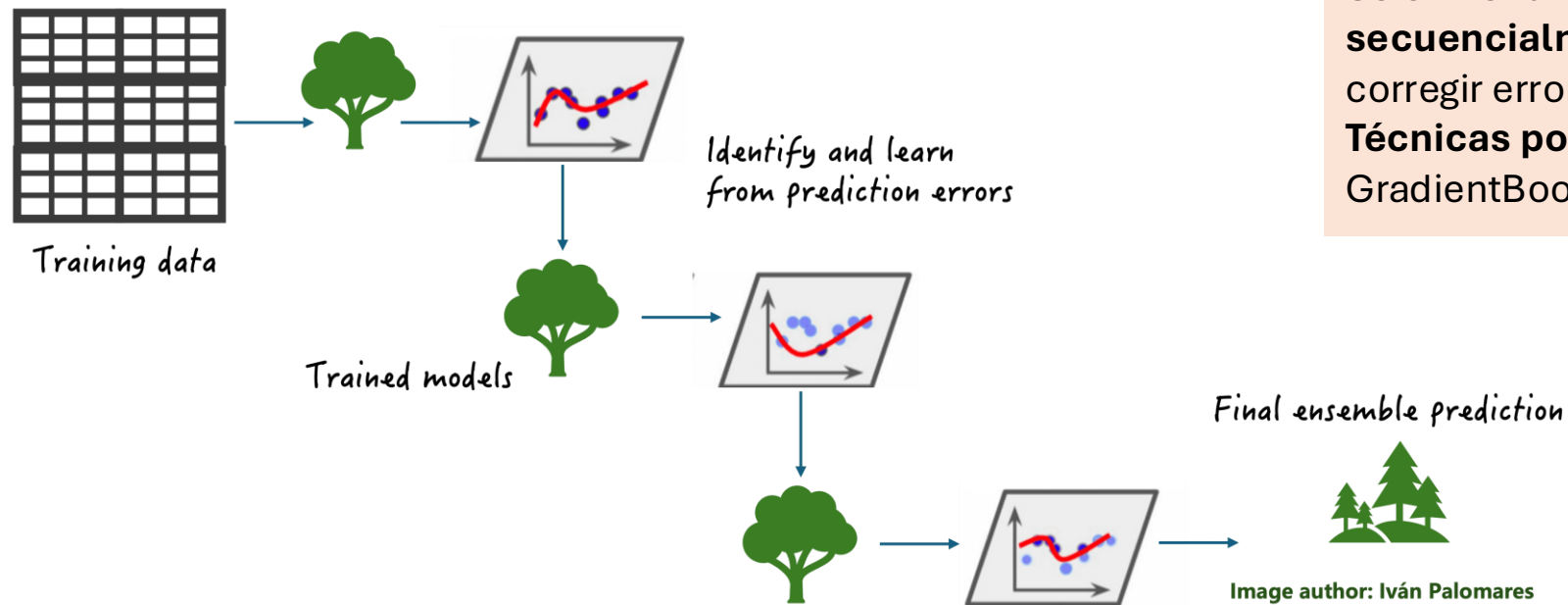
- Clasificación: voto por mayoría
- Regresión: promedio

Técnica popular: Random Forests

Ensembles

Combinar múltiples modelos en una sola tarea predictiva

Boosting Ensemble



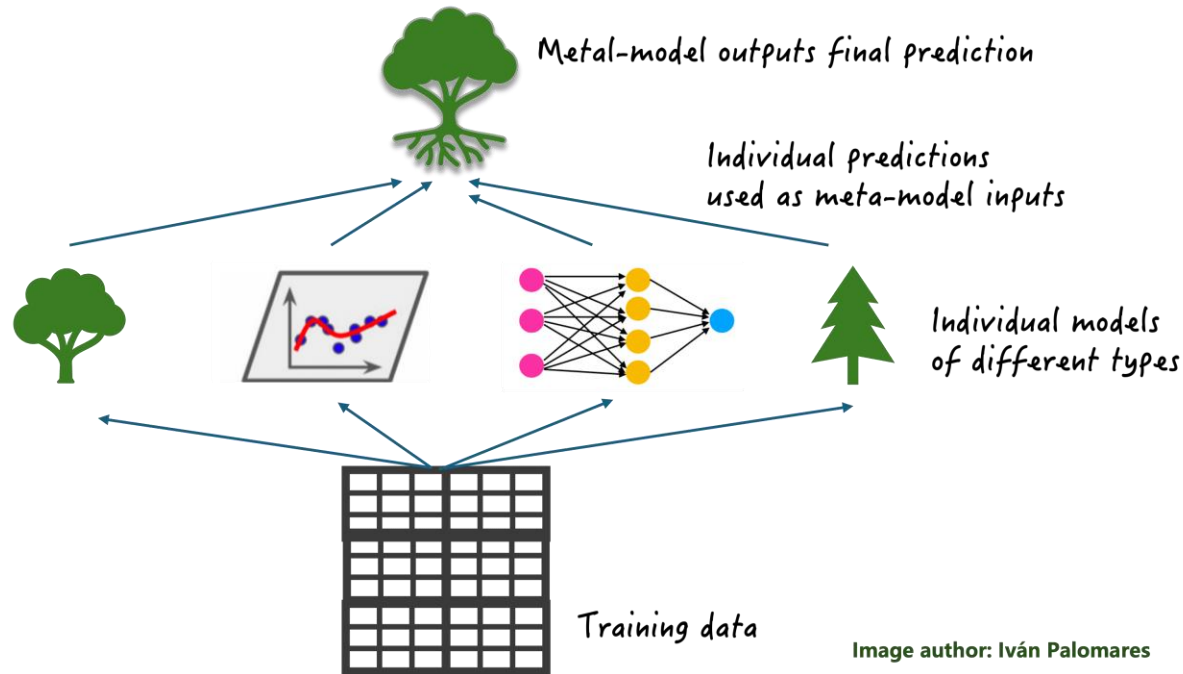
BOOSTING

Se entrenan varios modelos **secuencialmente**, cada modelo busca corregir errores predictivos del anterior.
Técnicas populares: XGBoost, GradientBoosting

Ensembles

Combinar múltiples modelos en una sola tarea predictiva

Stacking Ensemble



STACKING

Se entrenan **modelos de diferentes tipos en paralelo**.

Las **predicciones individuales** se usan como **variables de entrada para un meta-modelo** que las combina en una predicción final.

Meta-modelos comúnmente usados: regresión lineal, regresión logística

Ajuste de hiperparámetros en modelos de ML

Supón que hemos entrenado diferentes tipos de modelos a partir de un mismo dataset, y uno de ellos parece prometededor.

Tipo de modelo	Accuracy
kNN	0.75
Softmax	0.82
SVM	0.86
Decision Tree	0.85
Random Forest	0.92

¿Deberíamos aceptarlo sin más como nuestra solución final?

Ajuste de hiperparámetros en modelos de ML

Supón que hemos entrenado diferentes tipos de modelos a partir de un mismo dataset, y uno de ellos parece prometedor.

¿Deberíamos aceptarlo sin más como nuestra solución final?

Podríamos, pero estaríamos omitiendo la posibilidad de encontrar una *mejor configuración* para ese mismo modelo.

Ejemplo de hiperparámetros configurables en un *Random Forest*

- Relativos a los árboles de decisión:
 - Profundidad máxima, mínimo número de instancias en un nodo del árbol, ...
- Relativos al ensemble al completo, para controlar su estructura
 - Número de modelos (predictors) a entrenar, tamaño de muestras aleatorias, ...

Ajuste de hiperparámetros en modelos de ML

- El proceso de buscar una óptima configuración de los hiperparámetros se conoce como ***model fine-tuning***

Tipo de modelo	Accuracy
kNN	0.75
Softmax	0.82
SVM	0.86
Decision Tree	0.85
Random Forest	0.92



Configuración actual

max_features=3,
min_samples_split=10,
n_estimators=5

Opciones de búsqueda

max_features
= {1, 2, 3, 4, ...}

Min_samples_split
= {1, 2, ..., 10, ...,
20, ... 50, ...}

n_estimators =
= {1, ..., 50, ...
10, ...}

- En técnicas más sofisticadas como los ensembles, ¡esto puede suponer buscar una aguja en un pajar!

Ajuste de hiperparámetros en modelos de ML

Búsqueda en rejilla (grid search)

- Se explora automáticamente un subespacio de valores de varios hiperparámetros definidos a priori, para encontrar la mejor combinación posible.
- Los métodos de búsqueda emplean una **evaluación por validación cruzada** para evaluar cada posible combinación.

```
max_features  
= {1, 2, 3, 4, ...}
```

```
Min_samples_split  
= {1, 2, ..., 10, ...,  
20, ... 50, ...}
```

```
n_estimators =  
= {1, ... , 50, ...  
10, ...}
```

¿En qué consiste la validación cruzada (cross validation, CV)?

Cuando se ajusta un modelo, y antes de evaluarlo en un conjunto de datos de test, es una buena práctica particionar aún más el conjunto de entrenamiento, dejando aparte una porción llamada **conjunto de validación**.



Ajuste de hiperparámetros en modelos de ML

Búsqueda en rejilla (grid search)

- Se explora automáticamente un subespacio de valores de varios hiperparámetros definidos a priori, para encontrar la mejor combinación posible.
- Los métodos de búsqueda emplean una **evaluación por validación cruzada** para evaluar cada posible combinación.

```
max_features  
= {1, 2, 3, 4, ...}
```

```
Min_samples_split  
= {1, 2, ..., 10, ...,  
20, ... 50, ...}
```

```
n_estimators =  
= {1, ... , 50, ...  
10, ...}
```

¿En qué consiste la validación cruzada (cross validation, CV)?

CV divide el conjunto de entrenamiento en n partes o *folds*, usando todas menos una para entrenar una configuración del modelo durante el proceso de búsqueda, y la restante para validación. Se realizan todas las posibles combinaciones de particionado posibles.



Entrenar y validar modelo 1



Entrenar y validar modelo 2

...



Entrenar y validar modelo n

Ajuste de hiperparámetros en modelos de ML

Búsqueda aleatoria (random search)

- En vez de definir la “rejilla” manualmente, permitir una búsqueda más libre con combinaciones al azar.
- Funciona mejor ante un espacio de búsqueda demasiado amplio.
 - Solo requiere especificar el número m de rondas de búsqueda o iteraciones,
 - Así como el número n de particiones para la validación cruzada

**Número de rondas de entrenamiento
totales = $m \times n$**

Scikit-learn implementa ambos métodos de búsqueda con las clases:

- *GridSearchCV*
- *RandomizedSearchCV*

Optimización de Modelos con Optuna

Lejos de la fuerza bruta en GridSearch, Optuna es una librería Python que opta por una búsqueda inteligente, mediante Optimización Bayesiana

- Cada nuevo "intento" dentro de un "estudio" (proceso completo de optimización) no es aleatorio → se analizan resultados de modelos construidos en previos intentos para deducir matemáticamente las zonas más prometedoras.
- Código abierto: democratiza el acceso a optimización avanzada de modelos complejos de modo más eficiente → adoptado como estándar en la industria reciente

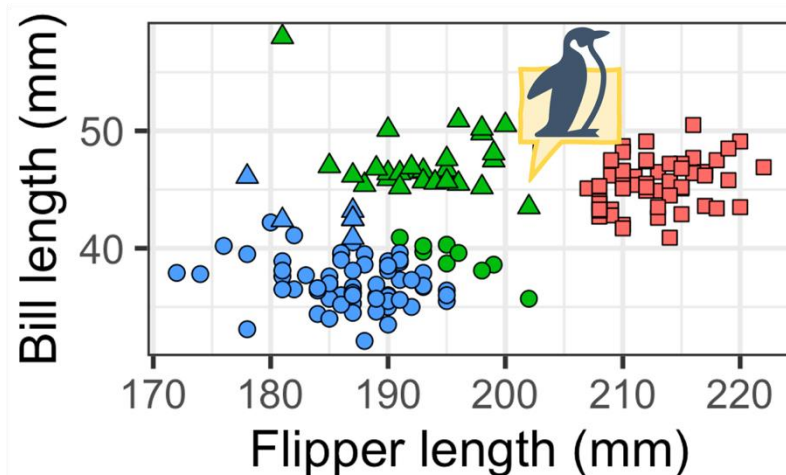
Optimización de Modelos con Optuna

```
def objective(trial):  
    # 1. Sugerir hiperparámetros de forma dinámica  
    params = {  
        'n_estimators': trial.suggest_int('n_estimators', 50, 200),  
        'max_depth': trial.suggest_int('max_depth', 3, 8),  
        # Usar escala logarítmica para el learning rate, una estrategia  
        # ideal para este parámetro  
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True)  
    }  
  
    # 2. Creamos y entrenamos el modelo con esos parámetros  
    model = GradientBoostingClassifier(**params, random_state=42)  
  
    # Hacemos una partición rápida del conjunto de train para validación interna  
    X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.2,  
                                                random_state=42)  
    model.fit(X_tr, y_tr)  
  
    # 3. Evaluar  
    preds = model.predict_proba(X_val)[:, 1]  
    return roc_auc_score(y_val, preds)
```

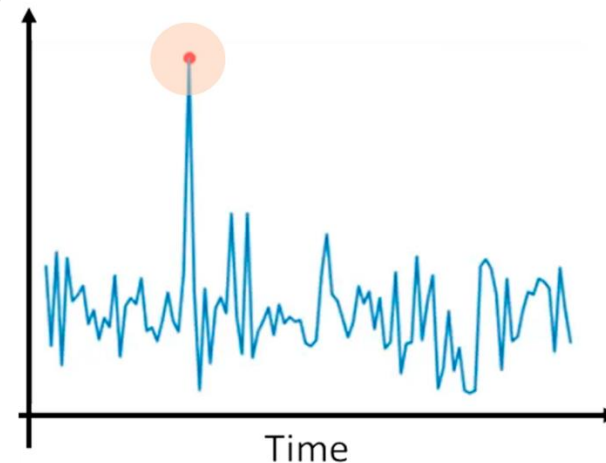
Aprendizaje no supervisado

- **A menudo, los datos históricos reales no están etiquetados.**
 - Los algoritmos de aprendizaje no supervisado extraen conocimiento y patrones de estos datos sin etiquetar.

Clustering: encontrar subgrupos con características similares



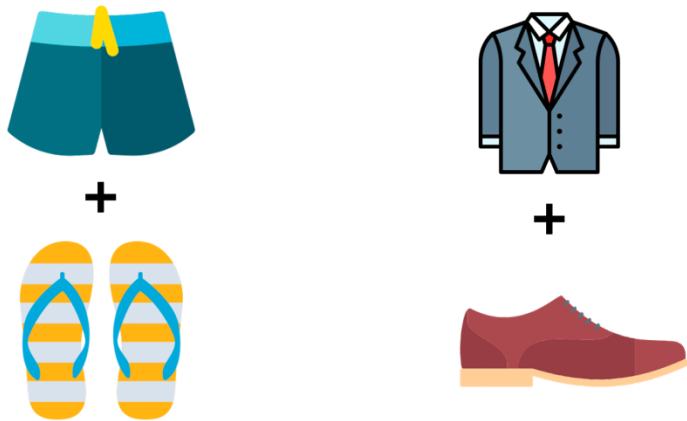
Detección de anomalías: identificar observaciones "inusuales", por ejemplo, transacciones bancarias sospechosas



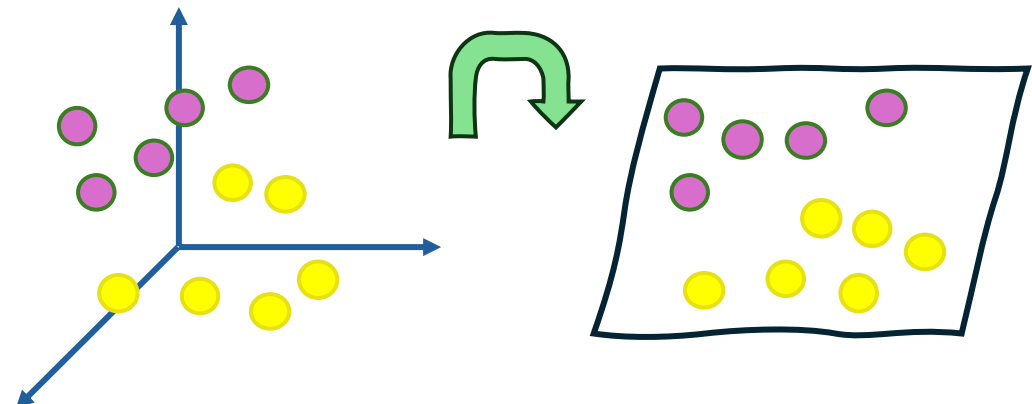
Aprendizaje no supervisado

- **A menudo, los datos históricos reales no están etiquetados.**
 - Los algoritmos de aprendizaje no supervisado extraen conocimiento y patrones de estos datos sin etiquetar.

Descubrimiento de reglas de asociación:
encontrar co-ocurrencias frecuentes en datos de caracter transaccional.

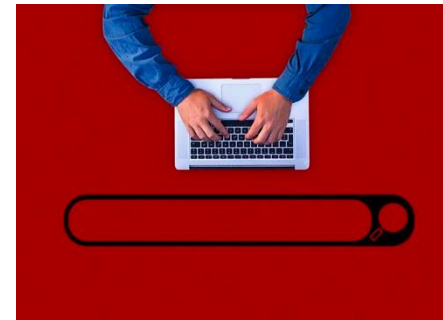
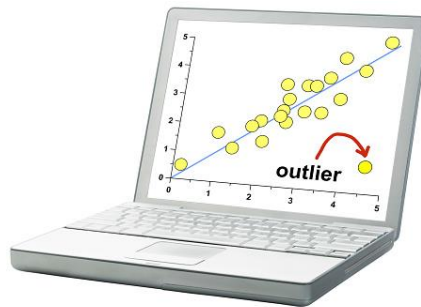


Reducción de la dimensionalidad: reducir número de atributos en los datos, manteniendo sus propiedades clave (útil p.ej. en datos de imágenes)



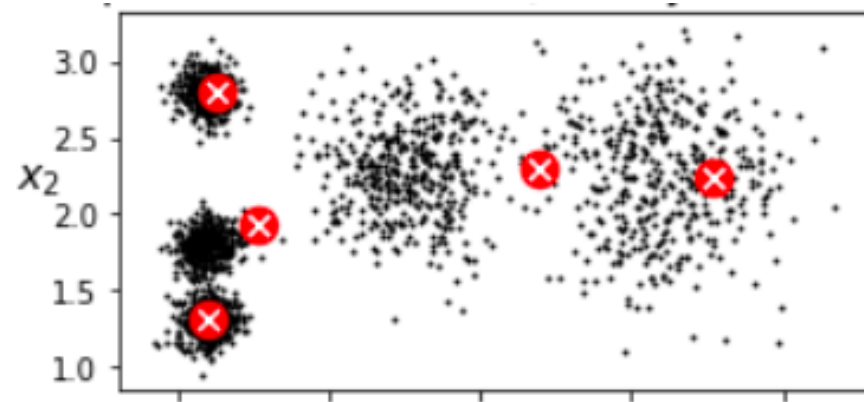
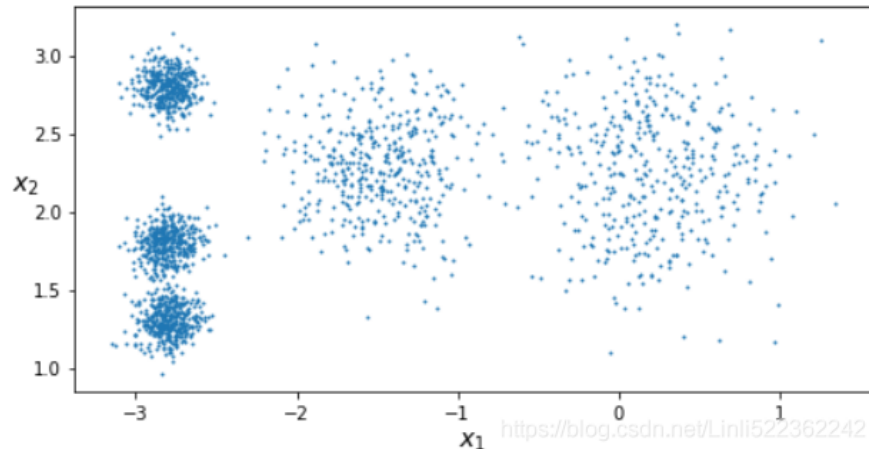
Clustering

- **Los procesos de agrupamiento son habituales en...**
 - Marketing: segmentación de clientes
 - Análisis exploratorio de los datos (EDA)
 - Detección de anomalías (*outliers*)
 - Motores de búsqueda: identificar documentos similares entre sí



El algoritmo K-means

- **Algoritmo iterativo originario del año 1957**
 - K-means agrupa los datos en k clusters, cada uno definido por un punto central llamado **centroide**
 - Requiere especificar el valor de k a priori, al igual que en clasificador KNN.



El algoritmo K-means

- **Ejemplo de clustering para segmentar clientes con K-means**



El algoritmo K-means

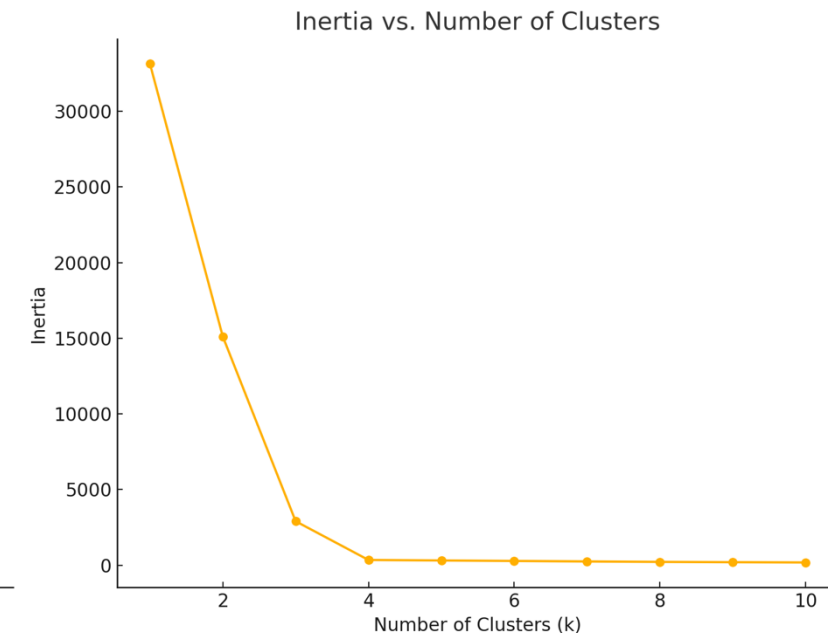
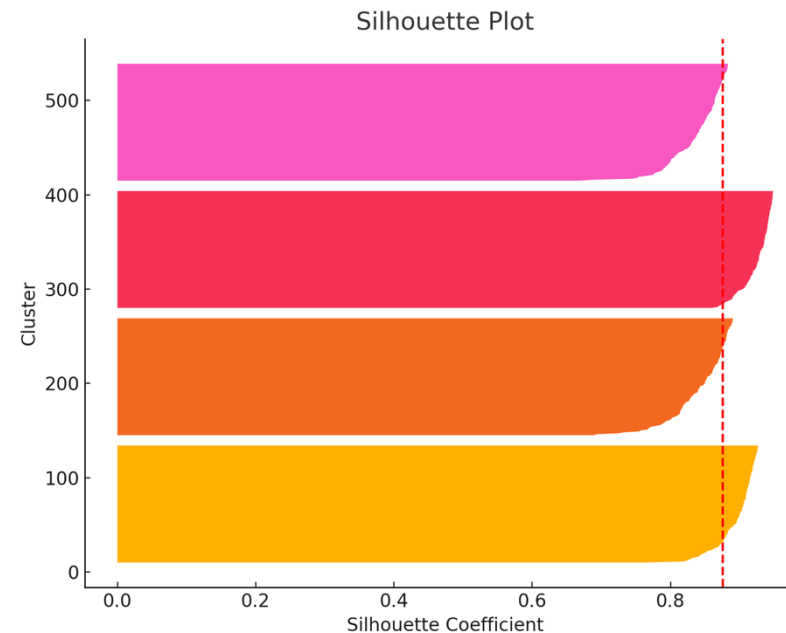
• Métricas de evaluación clusters

Silhouette Score

Mide cómo de separados están los clusters entre sí, y cómo de compactos son internamente. Se suele considerar bueno cuando es igual o mayor a 0.5

Inertia

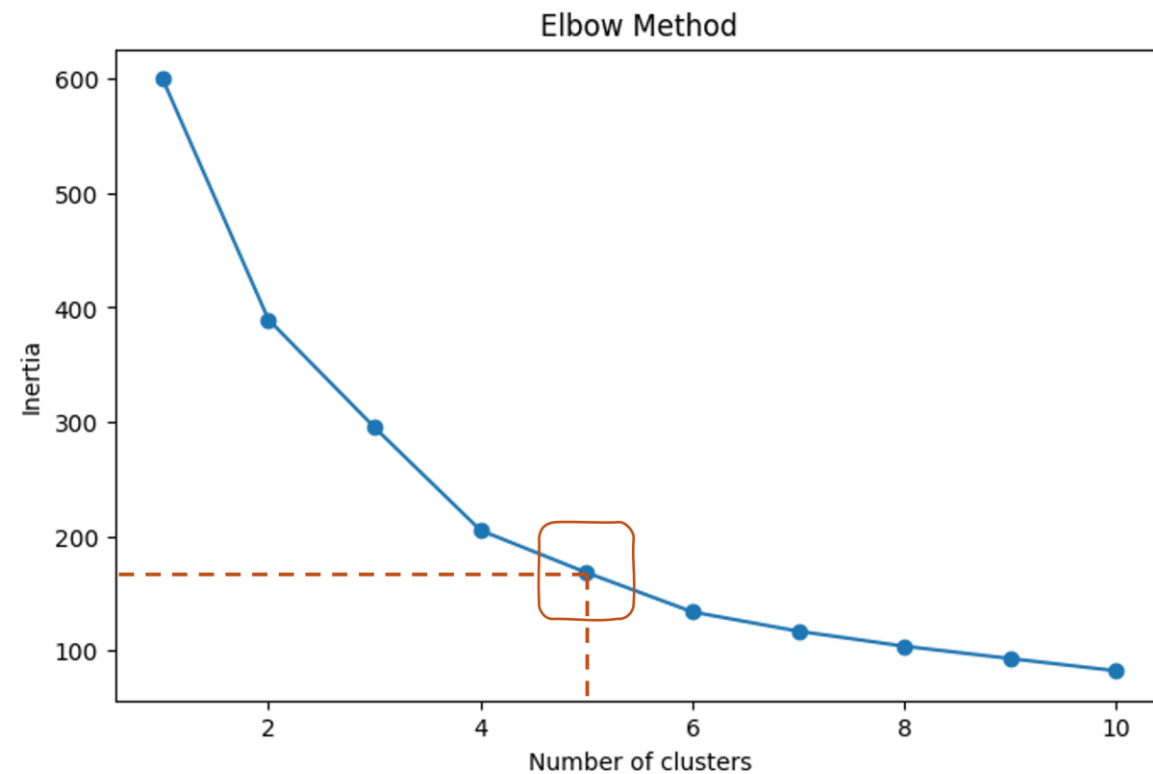
Mide compacidad. Suma de distancias cuadradas de cada punto a su centroide. Menor = mejor



El algoritmo K-means

- Una forma de decidir el número de clusters es el método del codo (Elbow Method)

Elegir k tal que tanto el propio k como la **inercia** del clustering resultante sean lo menores posibles (punto más cercano al origen de coordenadas)



Reducción de dimensionalidad: PCA

Algunos problemas a resolver mediante ML implican aprender de datasets que contienen **miles o millones de atributos** por instancia.

- Esto puede ralentizar notablemente el proceso de entrenamiento y también perjudicar el rendimiento del modelo resultante: **mal de la dimensionalidad**



El dataset **MNIST** es un típico ejemplo de datos de alta dimensionalidad

Con 70.000 instancias, cada una consiste en un imagen de **28x28 píxeles** de un dígito manuscrito del 0 al 9 (**10 clases**). Tiene por tanto $28 \times 28 = 784$ **atributos** in total.

https://en.wikipedia.org/wiki/MNIST_database

Reducción de dimensionalidad: PCA

Los datos de alta dimensionalidad también tienden a...



- **Ser más dispersos:** los ejemplos de entrenamiento suelen estar más alejados entre sí en un espacio multidimensional complejo
- Dar lugar a **predicciones menos fiables**.
- ¡Aumentan el riesgo de **overfitting**!
- Exigen un **número de instancias de entrenamiento muchísimo mayor** para que el modelo entrenado sea decente.

Reducción de dimensionalidad: PCA

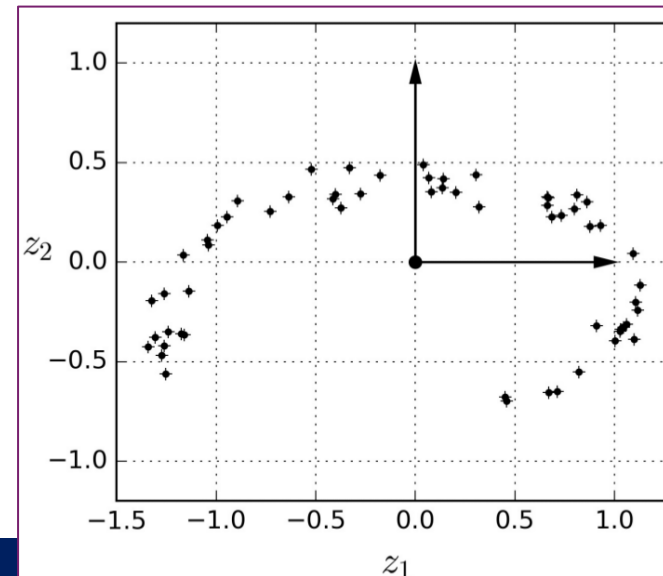
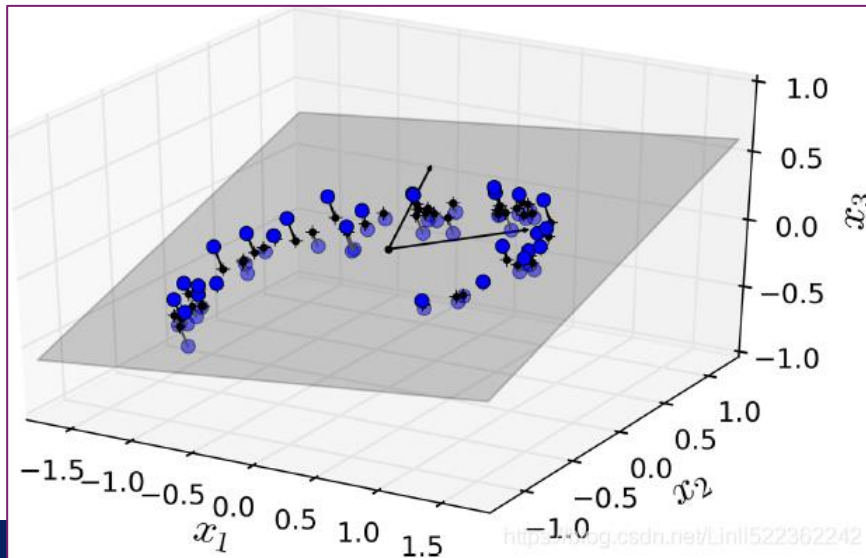
LA BUENA NOTICIA



- En la mayoría de situaciones, podemos **reducir drásticamente el número de atributos** con una mínima pérdida de información importante para que el modelo aprenda.
- **La pérdida de información no es necesariamente mala siempre**, a veces implica eliminar atributos ruidosos, redundantes o innecesarios que afectarían negativamente al poder predictivo del modelo
- **¡TAMBIÉN ES DE GRAN UTILIDAD PARA LA VISUALIZACIÓN DE DATOS COMPLEJOS!**

Reducción de dimensionalidad: PCA

- Debido a que:
 - (i) Algunos atributos apenas varían en comparación con otros, y
 - (ii) Existen a menudo correlaciones entre pares o subgrupos de atributos,
- Es posible a menudo obtener una representación o proyección bastante acertada de nuestros datos en un subespacio de menor dimensionalidad



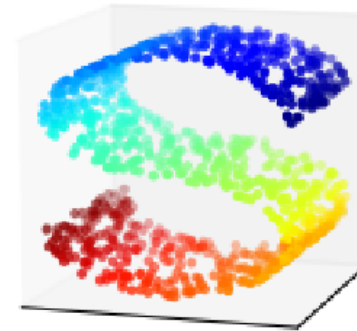
Reducción de dimensionalidad: PCA

Sin embargo, la proyección no siempre es la mejor solución en algunos datasets más desafiantes.

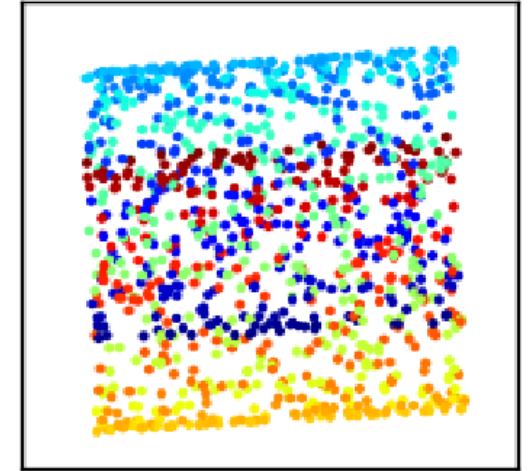
Hay técnicas más avanzadas como el llamado **Manifold Learning**:

- Retorcer o desdoblar la nube de puntos de los datos antes de proyectarlos en una dimensión menor

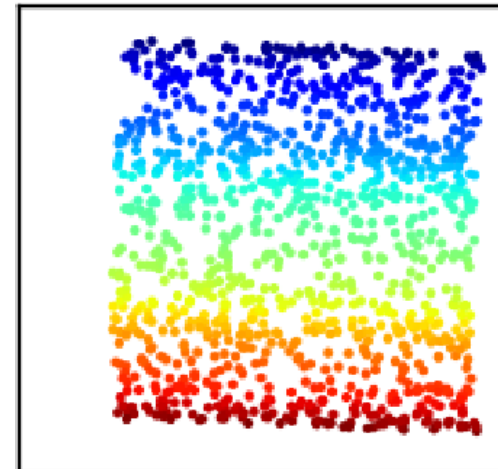
¡Echa un vistazo al famoso '[Swiss Roll](#)' dataset en google!



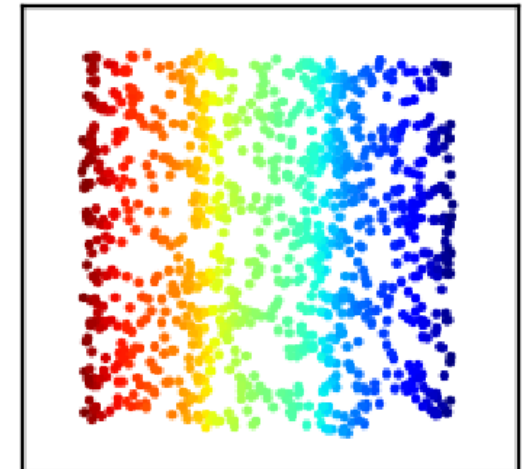
PCA projection



LLE projection



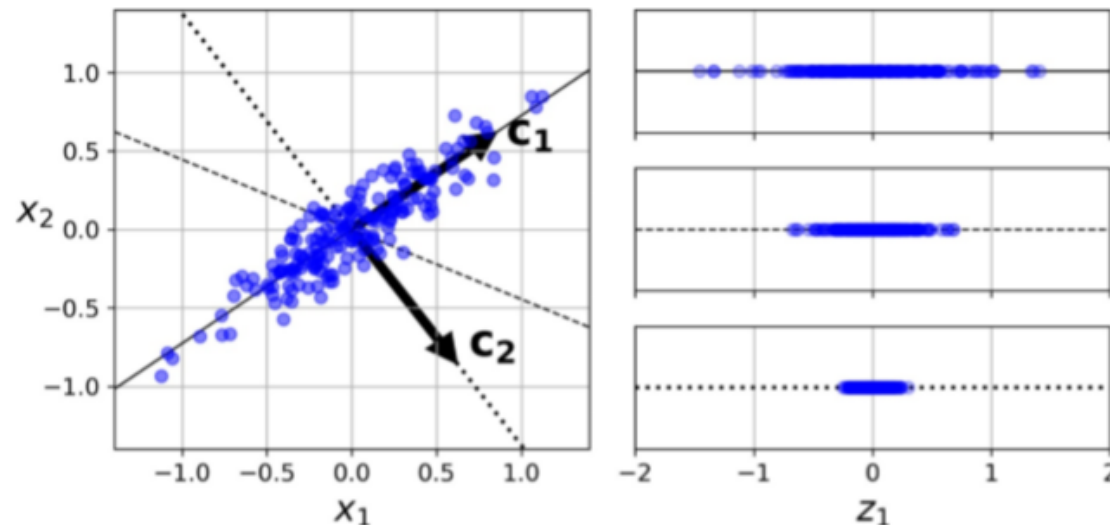
IsoMap projection



Reducción de dimensionalidad: PCA

PCA (Principal Component Analysis): popular técnica que hace una **proyección de los datos** en el hiperplano más cercano a estos.

Preservar tanta varianza como sea posible → menor pérdida de información.



- Mediante factorización de matrices -**Singular Value Decomposition (SVD)**- PCA identifica varios ejes ortogonales entre sí, que preservan la varianza → **Componentes Principales**